

实验一、mininet 应用实践

一、教学目的与学时建议

1. 能够运用 mininet 可视化工具创建计算机网络拓扑结构
2. 能够运用 mininet 交互界面创建拓扑结构
3. 能够运用 python 脚本构建计算机网络拓扑结构
4. 建议：2 学时

二、实验环境

1. 下载并安装虚拟机 VMware workstation;
2. 打开虚拟机镜像 (SDNHub_tutorial_VM_64-bit[NEW](可直接导入 VMware).ova),

下载链接: https://pan.baidu.com/s/1qYN_MtUboPmruHda1DgrTA 提取码: mhfi

注: 也可以在也可以安装其他虚拟机软件, 如 VirtualBox、KVM 等, 然后安装 mininet, 安装方法见 <http://mininet.org/download/>;

三、技术与知识背景

1. 什么是 Mininet?

Mininet 是由一些虚拟的终端节点 (end-hosts)、交换机、路由器连接而成的一个网络仿真器, 是一个轻量级软件定义网络和测试平台; 它采用轻量级的虚拟化技术使一个单一的系统看起来像一个完整的网络运行想过的内核系统和用户代码, 也可简单理解为 SDN 网络系统中的一种基于进程虚拟化平台, 它支持 OpenFlow、OpenvSwitch 等各种协议, Mininet 也可以模拟一个完整的网络主机、链接和交换机在同一台计算机上且有助于互动开发、测试和演示, 尤其是那些使用 OpenFlow 和 SDN 技术; 同时也可将此进程虚拟化的平台下代码迁移到真实的环境中。

Mininet 可以很方便地创建一个支持 SDN 的网络: host 就像真实的电脑一样工作, 可以使用 ssh 登录, 启动应用程序, 程序可以向以太网端口发送数据包, 数据包会被交换机、路由器接收并处理。有了这个网络, 就可以灵活地为网络添加新的功能并进行相关测试, 然后轻松部署到真实的硬件环境中。

2. Mininet 的特性:

- 1) 可以简单、迅速地创建一个支持用户自定义的网络拓扑，缩短开发测试周期
- 2) 可以运行真实的程序，在 Linux 上运行的程序基本上都可以在 Mininet 上运行，如 Wireshark
- 3) Mininet 支持 Openflow，在 Mininet 上运行的代码可以轻松移植到支持 OpenFlow 的硬件设备上
- 4) Mininet 可以在自己的电脑，或服务器，或虚拟机，或者云（例如 Amazon EC2）上运行
- 5) Mininet 提供 python API，简单易用
- 6) Mininet 是开源项目

3. Mininet 部分命令:

mininet > help	获取帮助列表。
mininet > nodes	查看 mininet 中结点的状态。
mininet > net	显示网络拓扑
mininet > dump	显示每个节点的接口设置和表示每个节点的进程的 PID
mininet > pingall	在网络中的所有主机之间执行 ping 测试
mininet > h1 ping h2	h1 和 h2 节点之间执行 ping 测试
mininet > h1 ifconfig	查看 host1 的 IP 等信息。
mininet > xterm h1	打开 host 1 的终端。
mininet > exit	退出 mininet 登录。

四、基础技能

1. 内容与任务

- 1) 用可视化工具 miniedit 创建一个网络拓扑，进行相应配置后，保存为 python 脚本，运行 python 脚本重现拓扑；
- 2) 命令行创建拓扑：
 - i. 最小的网络拓扑，一个交换机下挂两个主机。
 - ii. 每个交换机连接一个主机，交换机间相连接。本例：4 个主机，4 个交换机。
 - iii. 每个主机都连接到同一个交换机上。本例：3 个主机，一个交换机。
- 3) 交互式界面创建主机、交换机等
- 4) 对节点间进行 ping 测试。

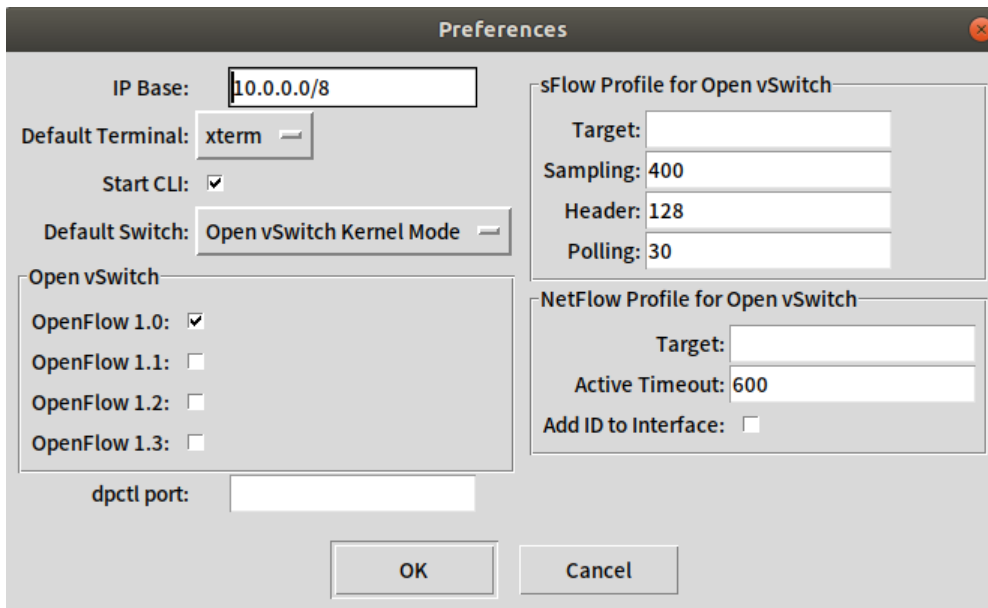
2. 规划与准备

在预习阶段，要求学生查阅相关书籍资料，就上述实验内容与任务进行分析。学生根据自己的掌握程度深入拓展。

五、基础技能的配置与操作要点

1. S1:

- 1) 获取超级用户权限后，进入到 mininet/examples 文件夹中，并运行其中的 miniedit.py 文件，打开可视化工具。
- 2) 在可视化工具中创建出简单的网络拓扑
- 3) 右击 host→“Properties”，选择对网络拓扑结构中的主机进行相关配置。
- 4) “Edit”→“Preferences”，勾选 strat CLI（允许通过交互界面对拓扑结构进行调整），选择所支持 Openflow 协议版本。



- 5) 保存为 python 文件，选择“File”→“Export Level 2 Script”。
- 6) 在终端运行保存下来的 python 文件即可。

2. S2:

- 1) 命令行: `sudo mn --topo single,2`
- 2) 命令行: `sudo mn --topo linear,4`
- 3) 命令行: `sudo mn --topo single,3`

3. S3:

- 1) 通过命令行“mn”，创建出最简单的拓扑结构也就是一个交换机下面直接连接两个主机，并且产生 mininet 的交互界面。
- 2) 输入“`py net.addHost('主机名')`”为拓扑结构添加主机。
- 3) 输入“`py net.addSwitch('交换机名称')`”为拓扑结构添加交换机。
- 4) 查看拓扑结构中存在的网络设备，观察设备是否被成功添加。

```

root@ubuntu:/home/keepdreams# mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> py net.addHost('h3')
<Host h3: pid=20450>
mininet> py net.addSwitch('s2')
<OVSSwitch s2: lo:127.0.0.1 pid=20453>
mininet> nodes
available nodes are:
c0 h1 h2 h3 s1 s2

```

4. S4:

- 1) pingall 查看所有主机之间的通信状况。
- 2) h1 ping -c4 h2 : h1 向 h2 发送四个包,测试 h1 与 h2 之间的通信

```

mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 X
h2 -> h1 X
h3 -> *** Error: could not parse ping output: connect: 网络不可达

X *** Error: could not parse ping output: connect: 网络不可达

X
*** Results: 50% dropped (2/4 received)
mininet>

```

六、进阶技能

1. 任务与要求

以前面的基本技能训练为基础，通过编写 python 脚本完成对于计算机网络拓扑结构的构建。

- 1) 编写 Python 脚本，定义如下拓扑：
 - i. topo linear,4

- ii. topo single,3
 - iii. topo tree,depth=2,fanout=2
 - iv. 1 个交换机, 2 个主机, 并且赋予主机 IP 地址。
- 2) 编写 Python 脚本, 对网络性能进行限制。观察下面给出的脚本文件, addHost()语法可以对主机 cpu 进行设置, 以百分数的形式; addLink()语法可以设置带宽 bw、延迟 delay、最大队列的大小 max_queue_size、损耗率 loss。

2. 提示与建议

1) S1:

- i. 针对每个小题所提出的拓扑结构进行相应深入的理解。
- ii. 通过基本技能训练中的以保存的 python 文件为基础进行学习。掌握 python 中各包括以下例举的语句作用及其用法。

```
s1 = net.addSwitch('s1', cls=OVSKernelSwitch, failMode='standalone')
```

```
h1 = net.addHost('h1', cls=Host, ip='10.0.0.2', defaultRoute=None)
```

```
net.addLink(h2, s1)
```

2) S2:

- i. addHost()使用示例: `h1 = self.addHost('h%s' % (h + 1),cpu=0.5)`
- ii. addLink()使用示例: `self.addLink(host, switch,bw=10, delay='5ms', loss=10, max_queue_size=1000, use_htb=True)`